Recurrent Neural Networks

Simple Class Notes — Beginner Friendly

Instructor: Ishteeaq Naeem · UMT Sialkot Campus

What will you learn?
✔ What is an RNN and why do we need it
✔ How RNN remembers things (hidden state)
✔ What is the vanishing gradient problem
✔ What is LSTM and GRU (simple idea)
✔ How to write RNN code in Keras (3 lines!)

1. The Problem — Normal Networks Forget Everything!

Imagine reading a book, but after every word you **completely forget** all previous words. You could never understand any sentence, right?

That is exactly how a normal (feedforward) neural network works. It only looks at the **current input** and has no memory of what came before.

Example:
Sentence: **'I grew up in France ... I speak fluent ___'**

The answer is **French**. But to know this, you need to remember **France** from many words ago. A normal network cannot do this — it forgets!

Where do we need memory?

Task	What we need to remember
Predict next word	All previous words in the sentence
Understand speech	Previous sounds / syllables
Translate a sentence	All words in the source language
Predict tomorrow's price	Past days' stock prices

■ **Solution = RNN!** RNN gives the network a **memory** so it can remember previous inputs while processing the current one.

2. What is an RNN? — Think of It Like a Loop

RNN stands for **Recurrent Neural Network**. The word *recurrent* just means **repeating**. The network runs the same step again and again for each word/value in the sequence.

Simple Real-life Analogy:

Think of an RNN like a student taking notes in class.

After every sentence the teacher says, the student:
1. Listens to the **new sentence** (current input)
2. Looks at their **notes so far** (memory / hidden state)
3. **Updates the notes** with new information

At the end, the student's notes contain information from the **whole lecture** — not just the last sentence!

How it looks step by step:

```
Word 1: 'I' → RNN reads it → Memory = ['I']
Word 2: 'love' → RNN reads it → Memory = ['I', 'love']
Word 3: 'Python' → RNN reads it → Memory = ['I', 'love', 'Python']

Final memory carries information from ALL words!
```

The hidden state = Memory

At every step, the RNN keeps a small vector called the **hidden state (h)**. This is its memory. It gets updated at every time step.

```
Step 1: h1 = RNN( x1, h0 ) ← first word, no memory yet
Step 2: h2 = RNN( x2, h1 ) ← second word, memory from step 1
Step 3: h3 = RNN( x3, h2 ) ← third word, memory from step 1 & 2

x = current input h = hidden state (memory)
```

■ The same RNN cell (same weights) is used at **every step**. It is like using the same brain to read each word — but the notes (hidden state) keep growing.

3. The Problem with Simple RNN — Short Memory!

Simple RNNs work fine for short sequences. But they have a big problem with **long sequences**.

> Imagine you read a 200-page book, but you can only hold **3 pages** worth of notes in your head. By the time you reach page 200, you have forgotten everything from page 1.
>
> Simple RNN has the same problem — it **forgets old information** in long sequences. This is called the **Vanishing Gradient Problem**.

Why does this happen? (Simple Explanation)

When training a neural network, we send error signals **backwards** through time to fix the weights. For a long sequence, this error signal has to travel through many steps.

Each step, the signal gets **multiplied by a small number**. After many steps, multiplying small numbers together gives an **extremely tiny number** — basically zero. So the network stops learning from old inputs.

```
Example: 0.9 × 0.9 × 0.9 × 0.9 × 0.9 × ... (50 times)
= 0.9^50 = 0.005 ← almost zero!

The error signal vanishes → network forgets old information
```

Problem	What Happens	Solution
Vanishing Gradient	RNN forgets long past — cannot learn long sequences	LSTM or GRU
Exploding Gradient	Error becomes too large — training becomes unstable	Gradient Clipping

4. LSTM — Long Short-Term Memory (Simple Idea)

LSTM was invented to solve the forgetting problem. It gives the network a **smarter memory system** with controls.

> Think of LSTM like a **smart notebook** with 3 buttons:
>
> ■ **Forget Button** — Cross out old information that is no longer needed
> ✍ **Write Button** — Write new important information
> ■ **Read Button** — Decide what information to show as output
>
> The network **learns** when to press each button. It learns to remember important things and forget unimportant things.

LSTM has TWO memories:

■ **Long-term memory** (Cell State) — stores important facts from far back

■ **Short-term memory** (Hidden State) — stores recent context

Real example: In the sentence **'I grew up in France ... I speak fluent ___'**

→ LSTM writes **France** into long-term memory when it reads it
→ Many words later, it still remembers France
→ When it reaches the blank, it reads France from memory
→ Output: **French** ✓

5. GRU — A Simpler Version of LSTM

GRU (Gated Recurrent Unit) is like LSTM but **simpler and faster**. Instead of 3 buttons, it has only **2 buttons**.

Feature	Simple RNN	LSTM	GRU
Memory buttons (Gates)	None	3 gates	2 gates
Speed	Fastest	Slow	Fast
Long sequence memory	Bad	Excellent	Good
Complexity	Simple	Complex	Medium
When to use	Never (for real tasks)	Complex NLP tasks	General use

■ **Which one to use?**
For beginners and most tasks → start with **LSTM**.
If you want faster training → try **GRU**.
Both are much better than Simple RNN for real problems.

6. RNN Code in Keras — Super Simple!

In Keras (Python), writing an RNN is just 3–4 lines. You only need to change ONE word to switch between RNN, LSTM, and GRU:

Step 1 — Import

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, LSTM, GRU, Dense
```

Step 2 — Build the Model

```
# Simple RNN
model = Sequential([
SimpleRNN(32, input_shape=(20, 1)), # 32 = memory size, 20 = sequence length
Dense(1) # output: predict 1 value
])

# Switch to LSTM — just change SimpleRNN to LSTM!
model = Sequential([
LSTM(32, input_shape=(20, 1)),
Dense(1)
])

# Switch to GRU — just change LSTM to GRU!
model = Sequential([
GRU(32, input_shape=(20, 1)),
Dense(1)
])
```

Step 3 — Compile and Train

```
model.compile(optimizer='adam', loss='mse')

model.fit(X_train, y_train, epochs=20, batch_size=32)
```

Step 4 — Make Predictions

```
predictions = model.predict(X_test)
```

■ **input_shape = (sequence_length, features)**
sequence_length = how many past steps you give as input (e.g. 20 past days)
features = how many values at each step (e.g. 1 for price, 3 for RGB color)

7. Real Life Examples of RNN

Application	Input	Output
Auto-complete on phone	Words you already typed	Next word suggestion
Google Translate	English sentence	Urdu / French sentence
Siri / Alexa	Your voice (audio frames)	Text / Action
Stock price prediction	Past 30 days prices	Tomorrow's price
Spam detection	Email words in order	Spam / Not Spam
Music generation	Previous musical notes	Next note

8. Quick Summary — Everything in One Table

Concept	Simple Explanation
Feedforward Network	No memory — only sees current input, forgets everything
RNN	Has memory (hidden state) — passes info from step to step
Hidden State h	The memory box — updated at every time step
Vanishing Gradient	Simple RNN forgets old info in long sequences
LSTM	Smart memory with 3 gates — remembers long-range info
GRU	Simpler LSTM with 2 gates — faster, similar accuracy
input_shape	(sequence_length, features) — shape of your data
Dense(1) at end	Final output layer — gives you the prediction

Remember this simple flow:

1. You have a **sequence** (words, prices, sounds...)
2. RNN reads it **one step at a time**
3. After each step it updates its **memory (hidden state)**
4. At the end it uses the memory to make a **prediction**

LSTM and GRU do the same thing, but with a **smarter memory** that does not forget!

9. Practice Questions

Q1. Why can a normal (feedforward) network NOT understand a sentence? What is missing?

Q2. What is the hidden state in an RNN? Explain in simple words.

Q3. What is the vanishing gradient problem? Give a simple everyday example.

Q4. LSTM has 3 gates. Name them and explain what each one does in simple words.

Q5. What is the difference between LSTM and GRU? When would you use each?

Q6. In Keras, what do you change to switch from SimpleRNN to LSTM?